

Лабораторная работа №12

дисциплина: Архитектура компьютера

Безлепкина Татьяна Игоревна

Содержание

Цель работы	5
Задание	6
Теоретическое введение	7
Выполнение лабораторной работы	8
Выводы	11
Контрольные вопросы	12
Список литературы	16

Список иллюстраций

1	код скрипта backup_me для первого задания	8
2	процесс создания и запуска скрипта backup_me	8
3	код скрипта show_args для второго задания	8
4	запуск скрипта show_args с 15 аргументами	9
5	код скрипта myls для третьего задания	9
6	запуск скрипта myls и его работа	10
7	код скрипта count_by_ext для четвёртого задания	10
8	запуск скрипта count_by_ext и его работа	10

Список таблиц

Цель работы

Изучить основы программирования в оболочке ОС UNIX/Linux. Научиться писать небольшие командные файлы.

Задание

- Задача 1. Создать скрипт, который при запуске делает резервную копию самого себя. Скрипт должен скопировать свой исходный код в директорию backup в домашнем каталоге пользователя и заархивировать его одним из архиваторов (tar, zip или bzip2). Использование архиватора требуется изучить через справочную систему (man).
- Задача 2. Создать скрипт, который обрабатывает произвольное количество аргументов командной строки, в том числе превышающее десять. Скрипт должен правильно работать с любым числом переданных аргументов, например, последовательно распечатывать их значения.
- Задача 3. Создать скрипт, который является аналогом команды ls без использования самой команды ls и команды dir. Скрипт должен выдавать информацию о содержимом указанного каталога (или текущего, если каталог не задан) и выводить информацию о правах доступа к файлам этого каталога (тип файла, возможность чтения, записи, выполнения).
- Задача 4. Создать скрипт, который получает в качестве аргументов командной строки формат файла (например, .txt, .doc, .jpg, .pdf) и путь к директории. Скрипт должен вычислить и вывести количество файлов с указанным расширением в заданной директории.

Теоретическое введение

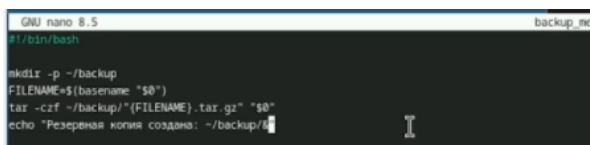
Командная оболочка (shell) — это программа, обеспечивающая взаимодействие пользователя с операционной системой через командную строку. Она интерпретирует команды, запускает программы, поддерживает переменные, управляющие конструкции (условия, циклы) и перенаправление ввода-вывода.

В ОС семейства UNIX/Linux наиболее распространены оболочки sh, bash, csh, ksh. Стандарт POSIX определяет единый набор интерфейсов для совместимости UNIX-подобных систем.

Bash (Bourne Again Shell) — это расширенная версия Bourne shell с поддержкой массивов, арифметических операций, функций и расширенных управляющих конструкций.

Выполнение лабораторной работы

Скрипт делает резервную копию самого себя (рис. 1)

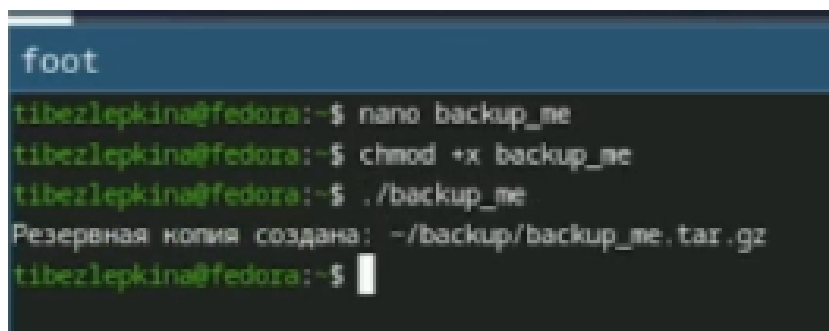


```
GNU nano 8.5 backup_me
#!/bin/bash

mkdir -p ~/backup
FILENAME=$(basename "$0")
tar -czf ~/backup/"$FILENAME".tar.gz "$0"
echo "Резервная копия создана: ~/backup/"
```

Рис. 1: код скрипта backup_me для первого задания

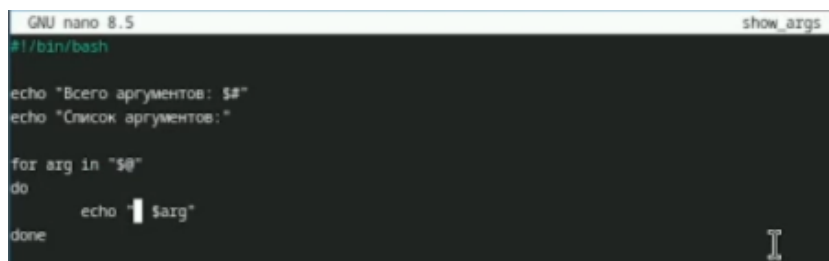
Создаю и запускаю скрипт backup_me (рис. 2)



```
foot
tibezelepkina@fedora:~$ nano backup_me
tibezelepkina@fedora:~$ chmod +x backup_me
tibezelepkina@fedora:~$ ./backup_me
Резервная копия создана: ~/backup/backup_me.tar.gz
tibezelepkina@fedora:~$
```

Рис. 2: процесс создания и запуска скрипта backup_me

Скрипт обрабатывает произвольное количество аргументов (рис. 3)



```
GNU nano 8.5 show_args
#!/bin/bash

echo "Всего аргументов: $#"
```

Рис. 3: код скрипта show_args для второго задания

Запускаю скрипт show_args, скрипт обрабатывает более 10 аргументов (рис. 4)

```
tibeziapkina@fedora:~$ nano show_args
tibeziapkina@fedora:~$ chmod +x show_args
tibeziapkina@fedora:~$ ./show_args 1 2 3 4 5 6 7 8 9 10 11 12 13 14 15
Всего аргументов: 15
Список аргументов:
1
2
3
4
5
6
7
8
9
10
11
12
13
14
15
```

Рис. 4: запуск скрипта show_args с 15 аргументами

Пишу код скрипта myls для третьего задания (рис. 5)

```
GNU nano 8.5 myls
#!/bin/bash

TARGET_DIR="${1:-.}"

if [ ! -d "$TARGET_DIR" ]; then
    echo "Ошибка: '$TARGET_DIR' не является каталогом"
    exit 1
fi

cd "$TARGET_DIR" || exit 1

echo "Содержимое каталога: $(pwd)"

if [ -z "$(echo *)" ] || [ "$(echo *)" = "" ]; then
    echo "(каталог пуст)"
    exit 0
fi
```

Рис. 5: код скрипта myls для третьего задания

Скрипт показывает содержимое каталога и права доступа (рис. 6)

```
tibezlepkina@fedora:~$ nano myls
tibezlepkina@fedora:~$ chmod +x myls
tibezlepkina@fedora:~$ ./mlys
Содержимое каталога: /home/tibezlepkina
-rw- abcl
drwx australia
drwx backup
drwx backup_images
-rwx backup_me
drwx bin
-rw- conf.txt
-rw- feathers
-rw- file.txt
-rw- #lab07.sh#
-rw- lab07.sh
-rw- may
```

Рис. 6: запуск скрипта myls и его работа

Пишу код скрипта count_by_ext для четвёртого задания (рис. 7)

```
GNU nano 8.5 count_by_ext
#!/bin/bash

if [ $# -ne 2 ]; then
    echo "Использование: $0 расширение директория"
    echo "Пример: $0 .txt /home/user/docs"
    exit 1
fi

EXT="$1"
DIR="$2"

if [ ! -d "$DIR" ]; then
    echo "Ошибка: директория '$DIR' не существует"
    exit 1
fi

cd "$DIR" || exit 1

count=0

if [ "$(echo *)" = "*" ]; then
```

Рис. 7: код скрипта count_by_ext для четвёртого задания

Скрипт правильно подсчитывает количество файлов с заданным расширением.
(рис. 8)

```
tibezlepkina@fedora:~$ nano count_by_ext
tibezlepkina@fedora:~$ chmod +x count_by_ext
tibezlepkina@fedora:~$ ./count_by_ext .txt ~/test_count
Количество файлов с расширением .txt в директории /home/tibezlepkina/test_count: 2
tibezlepkina@fedora:~$
```

Рис. 8: запуск скрипта count_by_ext и его работа

Выводы

В ходе лабораторной работы изучены основы программирования в bash: переменные, массивы, арифметика, управляющие конструкции (if, for, while), проверка прав доступа к файлам, обработка аргументов (в том числе более 10) и архивация.

Разработаны и протестированы четыре скрипта: - backup_me — резервная копия самого себя в ~/backup; - show_args — вывод любого количества аргументов; - myls — аналог ls (тип и права доступа файлов); - count_by_ext — подсчёт файлов по расширению. Все задания выполнены. Цель работы достигнута.

Контрольные вопросы

1. Объясните понятие командной оболочки. Приведите примеры командных оболочек. Чем они отличаются? Командная оболочка (shell) — это программа, которая принимает команды пользователя, интерпретирует их и передаёт ядру ОС для выполнения. Примеры: sh (Bourne shell) — базовая стандартная оболочка; bash (Bourne Again Shell) — расширенная версия sh с дополнительными возможностями; csh — C-подобная оболочка с историей команд; ksh (Korn shell) — объединяет свойства sh и csh. Отличаются синтаксисом, набором встроенных команд, поддержкой массивов, историей и совместимостью с POSIX.
2. Что такое POSIX? POSIX (Portable Operating System Interface for Computer Environments) — набор стандартов IEEE, описывающих интерфейсы между операционной системой и прикладными программами. Он обеспечивает совместимость различных UNIX-подобных систем и переносимость программ на уровне исходного кода.
3. Как определяются переменные и массивы в языке программирования bash? Переменная определяется как имя=значение (например, mark=/usr/andy/bin). Использование переменной — \$имя или \${имя}. Массив создаётся командой set -A имя значение1 значение2 ... (например, set -A states Delaware Michigan "New Jersey"). Добавление элемента: states[49]=Alaska. Обращение к элементу: \${states[0]}. Современный синтаксис: states=(Delaware Michigan "New Jersey").

4. Каково назначение операторов `let` и `read`? `let` используется для арифметических вычислений, например `let sum=x+7`. Ему не нужен знак `$` перед именами переменных. Альтернатива — двойные круглые скобки `((sum=x+7))`. `read` считывает данные со стандартного ввода и присваивает их переменным, например `read mon day trash` — первые два слова в `mon` и `day`, остальное в `trash`.
5. Какие арифметические операции можно применять в языке программирования `bash`? Сложение (+), вычитание (-), умножение (*), целочисленное деление (/), остаток от деления (%), возведение в степень (**), инкремент (++), декремент (--), битовые операции (&, |, ^, <<, >>), логические операции (&&, ||, !), операции сравнения (<, >, ==, !=, <=, >=).
6. Что означает операция `(( ))`? `(( ))` — это встроенная арифметическая конструкция `bash` для выполнения целочисленных вычислений. Внутри неё можно использовать переменные без знака `$`. Конструкция возвращает код завершения: 0, если результат не ноль, и 1, если ноль. Используется в условных выражениях и для присваивания, например `(( i++ ))` или `(( sum = a + b ))`.
7. Какие стандартные имена переменных Вам известны? `PATH` — список каталогов для поиска программ; `HOME` — домашний каталог пользователя; `IFS` — разделители полей (пробел, табуляция, перевод строки); `MAIL` — путь к файлу почты; `TERM` — тип терминала; `LOGNAME` — имя пользователя; `PS1` и `PS2` — приглашения командной строки; `PWD` — текущий каталог; `OLDPWD` — предыдущий каталог; `UID` — идентификатор пользователя; `SHELL` — путь к командной оболочке.
8. Что такое метасимволы? Метасимволы — это символы, имеющие специальный смысл для командного процессора: `*` (любая строка), `?` (один символ), `[ ]` (диапазон символов), `>` (перенаправление вывода), `<` (перенаправление ввода), `|` (конвейер), `( )` (экранирование), `&` (фон), `;`

- (разделитель команд), () (группировка), \$ (подстановка переменной), ' и " (кавычки).
9. Как экранировать метасимволы? Экранирование — это снятие специального смысла метасимвола. Способы: поставить перед метасимволом обратный слеш (например, * даст обычную звёздочку); заключить метасимволы в одинарные кавычки '...' (например, '*'); заключить в двойные кавычки "..." (экранируются все метасимволы, кроме \$, ', , ").
10. Как создавать и запускать командные файлы? Создать текстовый файл с первой строкой `#!/bin/bash` (shebang). Записать в него команды. Дать файлу право на выполнение командой `chmod +x имя_файла`. Запустить файл можно как `./имя_файла` или `bash имя_файла`.
11. Как определяются функции в языке программирования bash? Функция определяется с помощью ключевого слова `function` или без него: `function имя { список_команд; }` или `имя() { список_команд; }`. Например: `function clist { cd $1; ls; }`. Удалить функцию можно командой `unset -f имя`.
12. Каким образом можно выяснить, является файл каталогом или обычным файлом? С помощью команды `test` или её сокращённой формы `[ ]`. `[ -d "путь" ]` возвращает истину, если путь является каталогом. `[ -f "путь" ]` возвращает истину, если путь является обычным файлом. Также используются `[ -r ]`, `[ -w ]`, `[ -x ]` для проверки прав доступа.
13. Каково назначение команд `set`, `typeset` и `unset`? `set` — отображает текущие переменные и функции окружения; с флагом `-A` создаёт массив. `typeset` — задаёт свойства переменных и функций: `-f` показывает функции, `-ft` включает трассировку, `-fx` экспортирует функции, `-fu` делает функцию автоматически загружаемой. `unset` — удаляет переменную или функцию (с флагом `-f`).

14. Как передаются параметры в командные файлы? Параметры передаются при запуске скрипта через пробел: `./скрипт арг1 арг2 арг3`. Внутри скрипта доступны: `$1`, `$2`, ... `$9` — позиционные параметры; `$0` — имя скрипта; `$#` — количество параметров; `$*` и `@` — *shift()* *for arg in* "@".
15. Назовите специальные переменные языка `bash` и их назначение. `$0` — имя скрипта; `$1`–`$9` — аргументы скрипта; `$#` — количество аргументов; `$*` и `@` — все аргументы (строка или массив); `$?` — код возврата последней выполненной команды (0 — успех, не 0 — ошибка); `$$` — PID текущего процесса; `$_` — PID последней фоновой команды; `$-` — текущие флаги командного процессора; `${#var}` — длина строки в переменной; `${var:-word}` — значение по умолчанию; `${var:=word}` — присвоение по умолчанию; `${var:?word}` — вывод ошибки, если переменная не определена; `${var:+word}` — подстановка, если переменная определена.

Список литературы

1. Малахов С.В., Якупов Д.О. Принципы работы операционной системы Linux. Bash-скрипты: учебное пособие. Самара: ПГУТИ, 2024. 134 с